

SyncFlow Project Report

Durable AI Workflow Builder for Approval-Critical Enterprise Workflows

Product thesis, shipped implementation, architecture choices, QA results, and project learnings.

Sandip Pathe - June 16, 2026

Contents

Contents	2
Executive Summary	4
Final Product	4
Screenshots	4
Dashboard	4
Operator Builder	5
Completed Run Report	6
What Shipped	6
Backend	6
Frontend	7
Docs	8
Architecture Decisions	8
Local mode is explicit	8
Temporal remains the reliability story	8
One frontend event path	8
Approval rejection must be explicit	9
Product Learnings	9
Narrow beats broad	9
The job artifact needs a thesis	9
Demoability is a feature	9
UI copy should respect scope	9
Reports are better than raw canvas after completion	9
Engineering Learnings	9
Avoid duplicate event handling	9

Deterministic outputs make tests useful	10
Approval APIs should be truly async-safe	10
Existing schema was enough	10
QA needs negative paths	10
Design Learnings	10
Operator UI beats hackathon UI	10
Status should live on the node	10
Controls should be familiar	10
The right panel should answer the current question	10
QA Summary	11
Known Boundaries	11
What I Would Build Next	12
Files To Review First	12
Final Positioning	12

Author: Sandip Pathe Date: June 16, 2026 Branch: codex/syncflow-proof-of-work Pull request: sandip-pathe/syncflow#1

Executive Summary

SyncFlow is a proof-of-work for the AI workflow builder category. The project started as an older orchestration prototype and was reshaped into a focused hiring artifact for VectorShift-style engineering review.

The core thesis is simple:

Visual AI builders make workflow creation easy. The hard part is trust:

durable execution, human approvals, eval gates, audit trails, local

demoability, and operator clarity.

This is intentionally not a broad marketplace or a VectorShift clone. It is a sharp exploration of the reliability layer underneath no-code AI workflow builders.

Final Product

The reviewer path is the Private Market Diligence Review workflow:

evidence.

- Paste diligence notes or upload a text-like file.
- Extract key claims, risks, numbers, and assumptions.
- Run an eval gate for completeness and confidence.
- Pause for human approval.
- Generate an investment committee memo section.
- Show a final report with approval, audit, cost, latency, eval score, and node

The demo runs locally without Temporal, Redis, OpenAI, Docker, or cloud credentials. The production-style Temporal path remains available behind `EXECUTION_BACKEND=temporal`.

Screenshots

Dashboard

SYNCFLOW

Durable AI Workflow Builder

Docs API + New Workflow

Local demo ready Temporal path preserved

Open Demo →

Approval-critical AI workflows, shown through diligence.

Run a private-market diligence review from pasted notes, pause for human approval, evaluate risk completeness, and generate an audit-ready IC memo.

Backend
Local
No Docker or API keys

Eval gate
0.88
Risk completeness

Review
HITL
Approval before memo

Audit
Traceable
Cost, latency, nodes

Flagship Workflow
The hiring demo path reviewers should try first.
[Template](#)

Private Market Diligence Review [Template](#) [Recommended](#)
Extract claims, flag risks, require human approval, and draft an IC memo with audit-ready execution metadata. [Ready to run →](#)

Demo Run Story

- Paste or upload notes
- Extract claims and risks
- Score completeness
- Approve flagged risks
- Review final IC report

Review Notes

This demo intentionally uses deterministic local outputs. The product thesis is reliability, approvals, and auditability under AI workflow builders.

Workflow Library
Templates and session workflows available in this local demo. [4 workflows](#)

AI Content Moderation [Template](#)
AI screening with human approval for questionable content [Ready to run →](#)

Multi-Agent Research Team [Template](#)
Three AI agents collaborate on research: Researcher, Analyst, Writer [Ready to run →](#)

Operator Builder

BUILDER

Workflow Library
Drag nodes onto the canvas.

CORE NODES

- Trigger**
Start workflow
- AI Agent**
Call AI model
- API Call**
HTTP/API call
- Conditional**
If/else branching logic
- End**
End of workflow

TRUST NODES

- Approval**
Simple in-app confirmation
- Evaluation**
Policy check
- Merge**
Combine branches
- Event**
Pub/sub
- Timer**
Delay/wait

Private Market Diligence Re... Local Demo Run

DAG Mode

Diligence Notes
TRIGGER

Extract Claims & Risks
AGENT

Risk Completeness Eval
EVAL

Partner Review
APPROVAL

Draft IC Memo
AGENT

Audit-Ready Memo
END

+ - ↺ 🔒

Durable AI Workflow Builder for Approval-Critical Enterprise Workflows

Page 5

Completed Run Report

Private Market Diligence Review Report

IC memo, extracted diligence signals, approval decision, and audit metadata from the completed workflow run.

Investment Committee Memo

IC memo draft: The company shows strong growth and credible enterprise demand, but diligence should focus on account concentration, retention cohort quality, and whether AI-driven expansion produces durable gross margin.

RECOMMENDATION

Proceed to partner review with focused follow-up.

KEY POINTS

- Growth is strong, but buyer concentration needs validation.
- Retention should be verified by cohort and contract type.
- AI automation economics need usage-based margin evidence.

Flagged Risks

Risk Item	Severity
Retention quality needs validation Source material references retention; reviewer should verify evidence.	MEDIUM
Margin assumptions need support Source material references margin; reviewer should verify evidence.	MEDIUM

Approval

Decision: **approved**

APPROVER: approvals@syncflow.local

COMMENT: No comment

Audit Trail

Trigger: Source material captured

Extract Claims: Extracted 1 diligence claims, 3 material risks, and 1 assumptions requiring reviewer confirmation. \$0.0042 840ms

Eval: Claims, risk flags, and approval context are

[Back to Canvas](#)

What Shipped

Backend

approve edge.

- Added `EXECUTION_BACKEND=local|temporal`.
- Made local mode the default reviewer path.
- Added `LocalWorkflowExecutor` for deterministic local workflow execution.
- Preserved the existing Temporal workflow start path.
- Extended execute responses in local mode with:
 - `execution_backend`
 - `events`
 - `output`
 - `pending_approval`

- Stored paused local state in existing `Execution.output_data`.
- Created `ApprovalRequest` records from local approval nodes.
- Resumed local execution from the approval API.
- Emitted canonical approval audit events:
- `approval.requested`
- `approval.granted`
- `approval.denied`
- Fixed the rejection path so a rejected approval does not silently follow the
- Added backend tests for:
- local completion
- local approval pause
- local approval resume
- rejection stop behavior
- Temporal mode delegation
- workflow validation errors

Frontend

closing.

canonical.

- Rebranded the application to SyncFlow.
- Reworked the dashboard around the proof-of-work story.
- Added a polished local run input modal.
- Added text-like file upload for TXT, Markdown, JSON, and CSV.
- Added approval modal behavior that waits for the API call to succeed before
- Added a completed/stopped report modal.
- Added one event normalization helper for local responses and WebSocket events.
- Accepted legacy approval event names while making `approval.requested`
- Added neutral workflow name generation in the form `Adjective Noun Number`.
- Removed diligence/investor-specific copy from blank workflow canvases.
- Redesigned builder nodes as compact operator UI:
- neutral surfaces

- restrained color accents
- status chips
- cost, latency, and score chips
- Added a mode-aware right inspector:
- design mode: selected node config
- run mode: execution timeline
- completed mode: report summary and metrics
- Added frontend tests for event normalization and workflow naming.

Docs

walkthrough.

- Updated the root README with product thesis, screenshots, setup, and demo
- Added docs/architecture.md.
- Added docs/local-demo-runbook.md.
- Added docs/api-and-events.md.
- Added docs/demo-script.md.
- Added this project report and PDF artifact.

Architecture Decisions

Local mode is explicit

Local mode is not hidden behind a clever abstraction. It is a concrete `LocalWorkflowExecutor` that reuses the existing workflow definition shape and runs deterministic node behaviors.

That made the code reviewable. A reviewer can inspect exactly what local mode does for trigger, agent, eval, approval, and end nodes.

Temporal remains the reliability story

The goal was not to replace Temporal. The goal was to make the demo runnable without infrastructure while preserving the path that proves durable execution matters.

Local mode is for review and fast demos. Temporal mode is still the production orchestration path.

One frontend event path

The frontend normalizes backend events before applying them to node state. This keeps local HTTP response events and Temporal WebSocket events from becoming two different UI systems.

The event helper is the seam that prevents future drift:

- normalize backend event
- append event
- update node status
- set approval state
- set final output/report state

Approval rejection must be explicit

QA found that rejected approvals still followed the approve path. That is a dangerous trust bug. If a workflow has no reject edge, local mode now stops the run and produces a failed report with the rejection decision and audit data.

The learning is bigger than this one bug: approval nodes need careful branching semantics. Human review is only meaningful if rejection cannot accidentally mean "continue."

Product Learnings

Narrow beats broad

The project became stronger when it stopped trying to look like a generic workflow marketplace. A single serious diligence workflow made the demo easier to understand and easier to evaluate.

The job artifact needs a thesis

"AI workflow builder" is crowded. "Trust layer for approval-critical AI workflows" is sharper. It tells the reviewer what to look for in the code and why Temporal, approvals, evals, and reports belong together.

Demoability is a feature

Local mode is not just developer convenience. It changes how a recruiter or engineer experiences the project. They can run it without asking for API keys or debugging infrastructure.

UI copy should respect scope

Diligence and investment memo copy belongs only in the diligence template. Blank workflows should feel generic and reusable. This matters because hardcoded demo language makes the product feel fake.

Reports are better than raw canvas after completion

After a run finishes, the user does not mainly need the workflow graph. They need the result, approval decision, eval score, cost, latency, and audit trail. The large report modal became the right primary surface for completed runs.

Engineering Learnings

Avoid duplicate event handling

The WebSocket path and local response path originally risked duplicating status mapping. Extracting event normalization into `frontend/lib/workflow-events.ts` kept the behavior coherent.

Deterministic outputs make tests useful

Local demo AI outputs are deterministic. That makes it possible to test approval pause/resume, eval scores, cost totals, and report output without flakey model calls.

Approval APIs should be truly async-safe

The approval modal originally closed before the parent mutation finished. The fixed behavior waits for the API call. If resume fails, the user still has the modal and can retry.

Existing schema was enough

The work avoided a database migration by storing local execution state in `Execution.output_data` and the current node in `Execution.current_node`. That kept the implementation small and reduced risk during a short proof-of-work.

QA needs negative paths

The approve path worked, but the reject path exposed the real bug. For approval-critical workflows, negative-path QA is not optional.

Design Learnings

Operator UI beats hackathon UI

The first canvas felt like a toy builder: oversized nodes, heavy black floating controls, and weak information hierarchy. The improved UI uses compact nodes, quiet surfaces, clear statuses, and an inspector that changes by mode.

Status should live on the node

The node itself needs to show whether it is idle, running, waiting for approval, completed, or failed. It should also expose small metadata chips when available.

Controls should be familiar

The toolbar moved toward predictable product controls: project name, backend badge, primary Run button, and icon-only secondary actions with tooltips.

The right panel should answer the current question

The inspector changed from a generic side panel into a mode-aware surface:

- building: what is selected?
- running: what happened?

- completed: what did the run produce?

QA Summary

Final verification passed:

- frontend: `npm run test` -> 6 passed.
- frontend: `npm run lint` -> clean.
- frontend: `npm run build` -> passed.
- backend: `.venv\Scripts\python.exe -m pytest` -> 10 passed, 1 skipped.
- `git diff --check` -> clean, with LF-to-CRLF warnings only.
- Browser smoke test:
- dashboard loads
- diligence template opens
- run pauses for approval
- approval resumes
- report modal appears
- console errors: 0

QA health improved from 88/100 to 96/100 after fixing the rejection path.

Known Boundaries

These were intentionally not built in the 2-day artifact:

- full marketplace
- RBAC/auth
- real VDR ingestion
- PDF/DOCX parsing
- external connectors
- multi-tenant hardening
- plugin ecosystem
- knowledge-to-workflow compiler
- full metrics dashboard repair beyond the demo needs

The README and run modal are honest about text-like uploads only.

What I Would Build Next

dump.

- Source-grounded document ingestion with citations.
- Eval suites for each workflow template.
- Workflow versioning and diff review.
- Approval policies by role and risk level.
- Replayable execution history for debugging.
- Cost and latency budgets per workflow.
- Template analytics showing where users edit, fail, or abandon workflows.
- Real PDF/DOCX parsing with traceable source spans.
- Production auth and tenant boundaries.
- A smaller set of high-quality enterprise templates, not a broad template

Files To Review First

File	Why it matters
README.md	Product thesis, screenshots, quick start
backend/app/services/local_execution.py	Local execution model and approval behavior
backend/app/api/workflows.py	Execute endpoint and backend switch
backend/app/api/approvals.py	Approval resume path
backend/app/services/templates.py	Flagship template definition
frontend/lib/workflow-events.ts	Shared event normalization
frontend/components/modals/workflow-report.tsx	Completed/stopped report UX
frontend/components/toolbar/top.tsx	Run flow entry point
frontend/components/sidebar/properties.tsx	Mode-aware inspector
docs/architecture.md	System design explanation

Final Positioning

SyncFlow is a compact artifact with a clear point of view:

AI workflow builders need more than nodes and connectors. They need a trust

layer that makes runs inspectable, approvable, evaluable, and repeatable.

That is the part this project demonstrates.